

```
!pip install opencv-contrib-python scikit-image
```

```
Requirement already satisfied: opencv-contrib-python in /usr/local/lib/python3.13/dist-packages (4.14.0.94)
Requirement already satisfied: scikit-image in /usr/local/lib/python3.13/dist-packages (0.25.2)
Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.13/dist-packages (from opencv-contrib-python) (2.1.3)
Requirement already satisfied: scipy>=1.11.4 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (1.16.3)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (2.37.4)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (2026.8.23)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (26.3)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.13/dist-packages (from scikit-image) (0.5)
```

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from skimage.feature import hog
from skimage import exposure
from google.colab import files
```

```
uploaded = files.upload()

filename = next(iter(uploaded))
img = cv2.imread(filename)

# Convert BGR to RGB
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(8,6))
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()
```

eyeliner.jpeg
eyeliner.jpeg (image/jpeg) - 51789 bytes, last modified: 26/08/2026 - 100% done
Saving eyeliner.jpeg to eyeliner.jpeg

Original Image



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8,6))
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```

Grayscale Image



```
sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(gray, None)

print("Number of Keypoints:", len(keypoints))
print("Descriptor Shape:", descriptors.shape)
```

```
Number of Keypoints: 950
Descriptor Shape: (950, 128)
```

```
sift_image = cv2.drawKeypoints(
    img_rgb,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

plt.figure(figsize=(10,7))
plt.imshow(sift_image)
plt.title("SIFT Keypoints")
plt.axis("off")
plt.show()
```

SIFT Keypoints



```
# Resize image for consistent HOG extraction
hog_image = cv2.resize(gray, (128, 128))

features, hog_visual = hog(
```

```
hog_image,
orientations=9,
pixels_per_cell=(8, 8),
cells_per_block=(2, 2),
visualize=True
)

print("HOG Feature Vector Length:", len(features))
```

HOG Feature Vector Length: 8100

```
hog_visual = exposure.rescale_intensity(
    hog_visual,
    in_range=(0, 10)
)

plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.imshow(hog_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(hog_visual, cmap="gray")
plt.title("HOG Visualization")
plt.axis("off")

plt.show()
```

Grayscale Image



HOG Visualization



```
print("SIFT Keypoints:", len(keypoints))
print("SIFT Descriptor Shape:", descriptors.shape)
print("HOG Feature Length:", len(features))
```

SIFT Keypoints: 950
SIFT Descriptor Shape: (950, 128)
HOG Feature Length: 8100

```
uploaded2 = files.upload()

filename2 = next(iter(uploaded2))
img2 = cv2.imread(filename2)

gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8,6))
plt.imshow(cv2.cvtColor(img2, cv2.COLOR_BGR2RGB))
plt.title("Second Image")
plt.axis("off")
plt.show()
```

Choose files | kajal.jpeg

kajal.jpeg(image/jpeg) - 43075 bytes, last modified: 26/08/2026 - 100% done
Saving kajal.jpeg to kajal (1).jpeg

Second Image



```
keypoints2, descriptors2 = sift.detectAndCompute(gray2, None)

print("Image 2 Keypoints:", len(keypoints2))
```

Image 2 Keypoints: 560

```
bf = cv2.BFMatcher()

matches = bf.knnMatch(descriptors, descriptors2, k=2)

# Lowe's ratio test
good_matches = []

for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)

print("Good Matches:", len(good_matches))
```

Good Matches: 175

```
match_image = cv2.drawMatches(
    img,
    keypoints,
    img2,
    keypoints2,
    good_matches,
    None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS
)

plt.figure(figsize=(15,8))
plt.imshow(cv2.cvtColor(match_image, cv2.COLOR_BGR2RGB))
plt.title("SIFT Image Matching")
plt.axis("off")
plt.show()
```

